Studio Nx

P1-07 가입 · 로그인 JS — 함수 설명 합본 (09~17)

2026-10-07

목차

1. ai-notes/javascript/P1-07 - 가입·로그인 JS/09_auth-ui_setFieldState.md
2. ai-notes/javascript/P1-07 - 가입·로그인 JS/10_auth-ui_initPasswordToggles.md
3. ai-notes/javascript/P1-07 - 가입·로그인 JS/11_auth-ui_initOAuthPlaceholders.md
4. ai-notes/javascript/P1-07 - 가입·로그인 JS/12_auth-ui_initAuthState.md
5. ai-notes/javascript/P1-07 - 가입·로그인 JS/13_auth-ui_initLoginForm.md
6. ai-notes/javascript/P1-07 - 가입·로그인 JS/14_home_initLoginModalFromHash.md
7. ai-notes/javascript/P1-07 - 가입·로그인 JS/15_join_initDuplicateChecks.md
8. ai-notes/javascript/P1-07 - 가입·로그인 JS/16_join_initPasswordRules.md
9. ai-notes/javascript/P1-07 - 가입·로그인 JS/17_join_initJoinForm.md

JS 09 — `setFieldState()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-101 · FR-102
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	08 `logout()` (P1-02 끝) · 10 `initPasswordToggles()`

1. 하는 일

입력칸 하나의 **상태**(초록 · 빨강 · 기본)와 아래 **문구**를 바꾼다.

호출	화면 (join.html 이메일 칸)
`setFieldState(field, 'ok', '✓ Available')`	밑줄 **초록**, 아래 초록 글 "✓ Available"
`setFieldState(field, 'error', 'This email is already in use.')`	밑줄 **빨강**, 아래 빨강 글. 화면 읽기 프로그램은 "잘못된 값" 으로 읽음
`setFieldState(field, null)`	밑줄 기본 색(보라), 문구는 처음 HTML 의 안내 글로 (join.html 은 빈 글자)

`field` 는 입력칸을 감싼 `.field` 요소다 (예: `#join-email-field`).

```
<div class="field" id="join-email-field">          ← is-ok / is-error 가 여기 붙음
  <input class="field__control" id="join-email" …>  ← aria-invalid="true" 가 여기 붙음
  <p class="field__msg" id="join-email-msg"></p>     ← 문구가 여기 들어감
</div>
```

2. 언제 쓰나

P1-07 의 다른 함수들이 칸마다 결과를 보여 줄 때 부른다. 가장 먼저 만든 이유다.

부르는 곳 (P1-07)	칸	예
`initDuplicateChecks()`	이메일 · 닉네임	Check → ✓ Available / This email is already in use.
`initPasswordRules()`	비밀번호 · 비밀번호 확인	Use only English letters, numbers and symbols. / Passwords don't match.
`initJoinForm()`	서버가 422 로 돌려준 칸	`details.nickname` → 닉네임 칸 빨강

색은 `forms.css` 가 정한다 — 이 함수는 **클래스와 속성만** 바꾼다.

붙이는 것	`forms.css` 가 하는 일
`.field.is-ok`	밑줄 · 문구 초록 (`--c-ok`)
`.field.is-error` + `aria-invalid="true"`	밑줄 · 문구 빨강 (`--c-error`)

3. 코드

import (파일 맨 위)

```
// (없음)
```

`auth-ui.js` 는 아직 import 가 없다 (다음 함수들에서 `api.js` · `auth.js` 를 가져올 예정). **이 함수는 import 가 필요 없다** — `querySelector` · `classList` · `dataset` 은 브라우저 기본 기능이다.

함수

```
export function setFieldState(field, state, message) {
  if (!field) return;
  const control = field.querySelector('.field__control');
  const msg = field.querySelector('.field__msg');

  field.classList.remove('is-ok', 'is-error');
  if (control) control.removeAttribute('aria-invalid');

  if (state === 'ok') {
    field.classList.add('is-ok');
  } else if (state === 'error') {
    field.classList.add('is-error');
    if (control) control.setAttribute('aria-invalid', 'true');
  }

  if (msg) {
    if (msg.dataset.hint === undefined) msg.dataset.hint = msg.textContent; // 처음 안내 글을 기억
    msg.textContent = message === undefined ? msg.dataset.hint : message;
  }
}
```

최신 문법으로 쓰면 (공부용 — 실제 코드는 위의 전통 문법)

```
export function setFieldState(field, state, message) {
  if (!field) return;
  const control = field.querySelector('.field__control');
  const msg = field.querySelector('.field__msg');

  field.classList.toggle('is-ok', state === 'ok');
  field.classList.toggle('is-error', state === 'error');
  if (state === 'error') control?.setAttribute('aria-invalid', 'true');
  else control?.removeAttribute('aria-invalid');

  if (msg) {
    msg.dataset.hint ??= msg.textContent;
    msg.textContent = message ?? msg.dataset.hint;
  }
}
```

최신 문법	실제 코드 (전통 문법)	뜻
`control?.setAttribute(…)`	`if (control) control.setAttribute(…)`	`control` 이 있을 때만

최신 문법	실제 코드 (전통 문법)	뜻
`msg.dataset.hint ??= msg.textContent`	`if (msg.dataset.hint === undefined) msg.dataset.hint = msg.textContent`	비어 있을 때만 넣기
`message ?? msg.dataset.hint`	`message === undefined ? msg.dataset.hint : message`	값이 없으면 대신 쓸 값 (`??` 는 `null` 도 "없음" 으로 봄)
`classList.toggle('is-ok', 조건)`	`remove` 후 조건에 맞으면 `add`	조건이 참이면 붙이고 거짓이면 뗌

문법 풀이

import 문법	뜻
`export function`	다른 파일(`join.js` 등)이 `import { setFieldState } from '../core/auth-ui.js'` 로 가져다 씀

식	뜻
`if (!field) return;`	`field` 가 없으면(`null` 등) 아무것도 하지 않고 끝냄
`field.querySelector('.field__control')`	`field` **안에서** 이 클래스를 가진 첫 요소. 없으면 `null`
`classList.remove('is-ok', 'is-error')`	두 클래스를 한 번에 뗌. 없어도 오류 없음
`setAttribute('aria-invalid', 'true')` · `removeAttribute(…)`	HTML 속성 붙이기 · 떼기
`msg.dataset.hint`	HTML 의 `data-hint="…"` 속성. 이 함수가 처음 부를 때 만들어 안내 글을 보관
`조건 ? A : B`	조건이 참이면 A, 아니면 B (오래된 전통 문법 — 삼항 연산자)
`message === undefined`	세 번째 인자를 **안 넘겼는지**. `''`(빈 글자)를 넘기면 빈 글자로 바꿈

4. 흐름

1. `field` 가 없으면 끝.
2. 상태 표시를 모두 지운다 (`is-ok` · `is-error` · `aria-invalid`).
3. `'ok'` 면 초록, `'error'` 면 빨강 + `aria-invalid`.
4. 문구를 바꾼다 — 넘긴 문구, 없으면 처음 안내 글.

5. 왜 이렇게

선택	이유
먼저 모두 지운 뒤 붙임	`ok` → `error` 로 바뀔 때 `is-ok` 가 남아 초록 · 빨강이 섞이지 않는다
색은 CSS 에, JS 는 클래스만	디자인을 바꿀 때 JS 를 건드리지 않는다. 같은 상태 규칙을 모든 폼(`forms.css`)이 쓴다
`aria-invalid="true"`	색을 볼 수 없는 사람도 화면 읽기 프로그램으로 "잘못된 입력" 을 안다. 문구는 `aria-live="polite"` 칸이라 바뀌면 읽어 준다
처음 안내 글을 `data-hint` 에 보관	"2–20 characters" 같은 안내가 있는 칸에서, 오류를 지우면 안내 글이 돌아온다. HTML 을 고치지 않아도 된다
`field` · `control` · `msg` 가 없어도 오류 없음	칸 모양이 조금 다른 폼(로그인 모달 등)에서도 안전하게 쓸 수 있다

6. 시험 (실제 join.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 11개)

P1-02 의 Node 시험과 달리, 이 함수는 실제 HTML · CSS 가 있어야 하므로 **브라우저 안에서** 시험했다 (Sentry 를 끈 8001 번 개발 서버).

시험	확인한 것
처음 상태	클래스 없음, `aria-invalid` 없음, 문구 빈 글자
ok · error · null · error→ok (4개)	클래스 · `aria-invalid` · 문구가 표대로, 바뀔 때 이전 상태가 남지 않음
실제 밑줄 색	error `rgb(255, 138, 138)` · ok `rgb(127, 224, 168)` · 기본 `rgba(163, 131, 255, 0.45)` — `forms.css` 가 실제로 적용됨
안내 글이 있는 칸 (4개)	error 뒤 null 이면 "2–20 characters" 로 복원, 문구 없이 ok 도 안내 글, 빈 글자를 넘기면 빈 글자
`field` 가 `null`	오류 없음

시험 중 한 번 실패한 것: 처음에는 밑줄 색이 세 상태 모두 같게 나왔다. `forms.css` 의 색 전환 효과(`transition`) 때문에 바꾼 직후에는 이전 색이었다. 함수 문제가 아니라 시험이 너무 빨리 읽은 것이라, 0.6초 기다린 뒤 읽도록 고쳤다.

11/11 통과.

JS 10 — `initPasswordToggles()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-101 · FR-103
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	09 `setFieldState()` · 11 `initOAuthPlaceholders()`

1. 하는 일

비밀번호 칸 오른쪽의 **Show / Hide** 버튼이 동작하게 한다. 누를 때마다 입력한 비밀번호를 보였다 가렸다 한다.

누르기 전	누른 뒤
`••••••••` · 버튼 **Show**	`Abcdef1!` · 버튼 **Hide**
`<input type="password">` · `aria-pressed="false"`	`<input type="text">` · `aria-pressed="true"`

입력한 값은 바뀌지 않는다. 보이는 방식(`type`)만 바뀐다.

2. 언제 쓰나

페이지가 열릴 때 한 번 부른다. 그러면 그 페이지의 Show 버튼이 모두 동작한다.

페이지	버튼	부르는 곳
join.html	Password · Confirm password · 로그인 모달 Password (3개)	`join.js`
home.html	로그인 모달 Password	`home.js` (P1-07 뒤 순서)
userInfo.html	현재 · 새 · 확인 비밀번호, 탈퇴 비밀번호 (4개)	`userinfo.js` (P1-10)

버튼과 입력칸은 HTML 의 `aria-controls` 로 짝지어져 있다.

```
<input class="field__control" id="join-password" type="password" …>
<button type="button" class="pw__toggle" aria-controls="join-password" aria-pressed="false">Show</button
>
```

3. 코드

import (파일 맨 위)

```
// (없음)
```

`auth-ui.js` 는 아직 import 가 없다. **이 함수는 import 가 필요 없다** — 브라우저 기본 기능만 쓴다.

함수

```
export function initPasswordToggles(root = document) {
  const buttons = root.querySelectorAll('.pw__toggle');
  for (const button of buttons) {
    if (button.dataset.pwToggle === 'on') continue;
    button.dataset.pwToggle = 'on';
    button.addEventListener('click', () => {
      const input = document.getElementById(button.getAttribute('aria-controls'));
      if (!input) return;
      const show = input.type === 'password'; // 지금 가려져 있으면 → 보이게
      input.type = show ? 'text' : 'password';
      button.textContent = show ? 'Hide' : 'Show';
      button.setAttribute('aria-pressed', show ? 'true' : 'false');
    });
  }
}
```

최신 문법으로 쓰면 (공부용 — 실제 코드는 위)

```
export function initPasswordToggles(root = document) {
  root.querySelectorAll('.pw__toggle:not([data-pw-toggle])').forEach((button) => {
    button.dataset.pwToggle = 'on';
    button.addEventListener('click', () => {
      const input = document.getElementById(button.getAttribute('aria-controls'));
      if (!input) return;
      const show = input.type === 'password';
      input.type = show ? 'text' : 'password';
      button.textContent = show ? 'Hide' : 'Show';
      button.ariaPressed = String(show);
    });
  });
}
```

최신 문법	실제 코드	뜻
`querySelectorAll('…:not([data-pw-toggle])')`	반복문 안에서 `if (… === 'on') continue;`	이미 연결한 버튼 건너뛰기
`.forEach((button) => { … })`	`for (const button of buttons) { … }`	하나씩 돌기
`button.ariaPressed = String(show)`	`button.setAttribute('aria-pressed', show ? 'true' : 'false')`	속성 바꾸기 (`ariaPressed` 는 2020년 무렵 생긴 속성 이름)

문법 풀이

import 문법	뜻
`export function`	`join.js` 등이 가져다 씀

식	뜻
`root = document`	인자를 안 주면 페이지 전체에서 찾음 (기본값)
`root.querySelectorAll('.pw__toggle')`	범위 안의 Show 버튼 **모두**
`for (const button of buttons)`	하나씩 꺼내 반복
`continue`	이번 차례를 건너뛰고 다음 버튼으로
`button.dataset.pwToggle = 'on'`	HTML 에 `data-pw-toggle="on"` 을 붙여 "연결함" 표시

식	뜻
`addEventListener('click', () => { … })`	누를 때마다 `{ … }` 를 실행
`getAttribute('aria-controls')`	버튼이 가리키는 입력칸의 `id`
`show ? 'text' : 'password'`	보일 차례면 `'text'`, 아니면 `'password'`

4. 흐름

1. 범위 안의 Show 버튼을 모두 찾는다.
2. 이미 연결한 버튼은 건너뛴다.
3. 각 버튼에 "누르면" 동작을 붙인다 — 짝 입력칸을 찾아 `type` · 글자 · `aria-pressed` 를 함께 바꾼다.

5. 왜 이렇게

선택	이유
같은 버튼에 두 번 연결하지 않음 (`data-pw-toggle`)	두 번 연결되면 한 번 누를 때 두 번 바뀌어 **아무 일도 안 일어난 것처럼** 보인다. 로그인 모달처럼 나중에 끼워 넣는 HTML 이 있어 함수가 여러 번 불릴 수 있다
`root` 인자	나중에 끼워 넣은 부분(모달 등)만 골라 연결할 수 있다
입력칸은 `aria-controls` 로 찾음	버튼 바로 앞 칸을 찾는 방식보다 HTML 구조가 바뀌어도 안전하다. 화면 읽기 프로그램도 같은 속성으로 짝을 안다
`aria-pressed` 도 함께	화면 읽기 프로그램이 "Show, 눌림" / "눌리지 않음" 으로 상태를 읽는다
글자 · 상태는 입력칸의 지금 `type` 을 보고 정함	버튼 글자를 기준으로 하면, 다른 코드가 `type` 을 바꿨을 때 어긋난다

6. 시험 (실제 join.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 9개)

`join.js` 가 페이지를 열 때 이미 이 함수를 부르므로, 시험에서 **두 번 더** 불러 중복 연결이 막히는지도 함께 확인했다.

시험	확인한 것
연결	페이지의 Show 버튼 3개(가입 2 + 로그인 모달 1)가 모두 연결됨
누르기	처음 `password · Show · false` → 한 번 누르면 `text · Hide · true` (세 번 불렀어도 **한 번만** 바뀜) → 다시 누르면 처음으로
값	입력한 `Abcdef1!` 은 그대로
짝	확인 칸 버튼은 확인 칸만 바꿈
`root`	새로 넣은 버튼은 연결 전, `initPasswordToggles(box)` 뒤에는 동작
짝 칸이 없는 버튼	눌러도 오류 없음, 글자 그대로

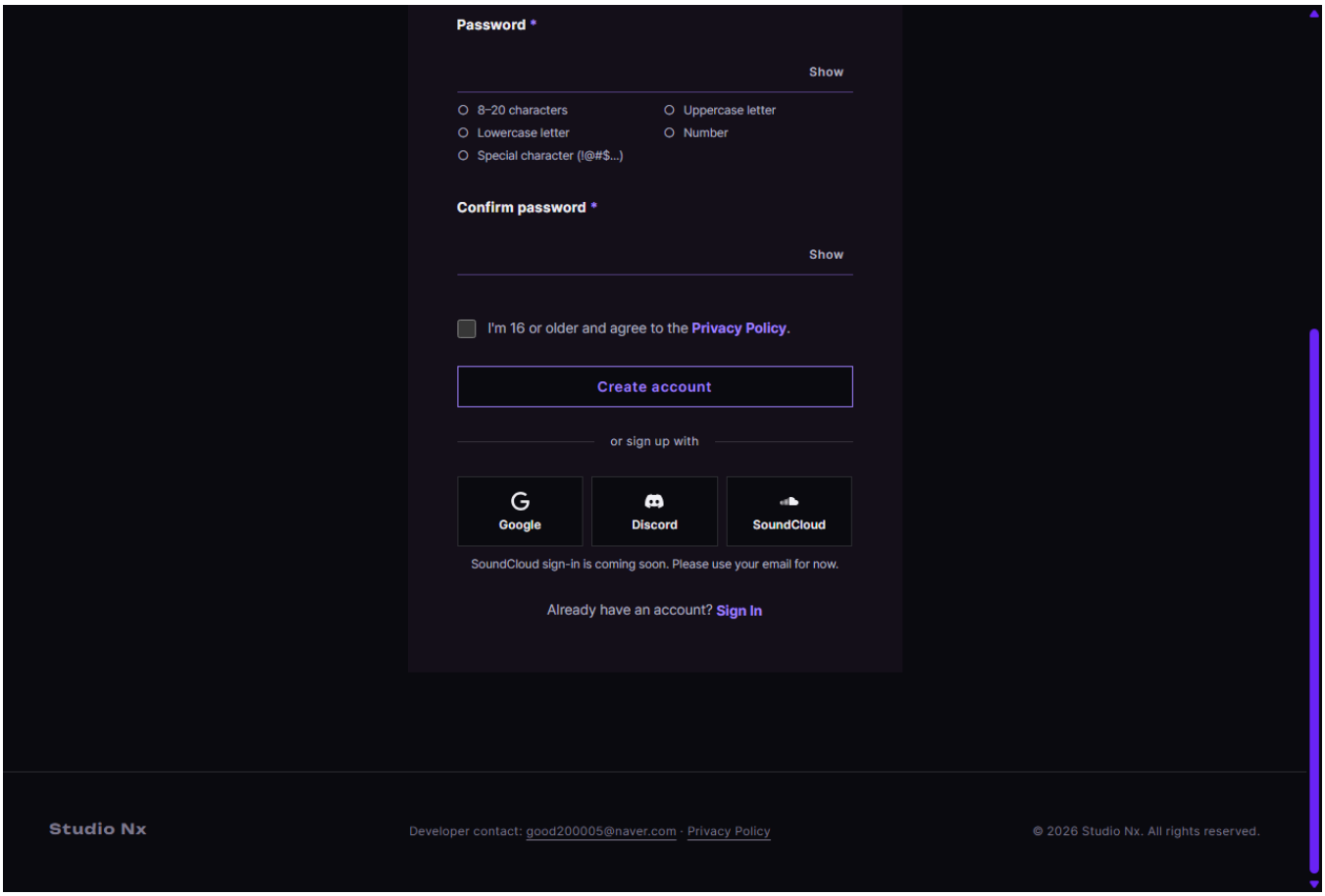
9/9 통과.

JS 11 — `initOAuthPlaceholders()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js` · `studionx/frontend/static/css/auth.css` (한 줄)
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-110
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	10 `initPasswordToggles()` · 12 `initAuthState()`

1. 하는 일

소셜 로그인 버튼(**Google · Discord · SoundCloud**)을 누르면, 버튼 줄 바로 아래에 "아직 준비 중" 안내 한 줄을 보여 준다. 연동은 나중이라(FR-110) 지금은 자리만 있다.



누른 버튼	버튼 아래 안내
Google	Google sign-in is coming soon. Please use your email for now.
Discord	Discord sign-in is coming soon. Please use your email for now.
SoundCloud	SoundCloud sign-in is coming soon. Please use your email for now.

다른 버튼을 누르면 **같은 줄의 글만 바뀐다** (줄이 늘어나지 않음).

2. 언제 쓰나

페이지가 열릴 때 한 번 부른다.

페이지	버튼 줄
join.html	가입 폼 아래 "or sign up with" 줄 + 로그인 모달 안 줄 (버튼 6개)
home.html	로그인 모달 안 줄 (`home.js`, P1-07 뒤 순서)

나중에 연동할 때: 이 함수 안의 "안내 쓰기" 부분을 `location.href = '/api/v1/auth/oauth/<provider>/start'` 로 바꾸면 된다 (템플릿 주석, PRD 01 개정 필요).

3. 코드

import (파일 맨 위)

```
// (없음)
```

이 함수는 import 가 필요 없다 — 브라우저 기본 기능만 쓴다.

함수

```
/** 소셜 로그인 서비스 이름 (안내 문구용) */
const OAUTH_NAMES = { google: 'Google', discord: 'Discord', soundcloud: 'SoundCloud' };

export function initOAuthPlaceholders(root = document) {
  const buttons = root.querySelectorAll('.oauth[data-provider]');
  for (const button of buttons) {
    if (button.dataset.oauthReady === 'on') continue;
    button.dataset.oauthReady = 'on';
    button.addEventListener('click', () => {
      const provider = button.dataset.provider;
      const name = OAUTH_NAMES[provider] || provider;
      const row = button.closest('.oauth-row') || button.parentElement;
      let note = row.nextElementSibling;
      if (!note || !note.classList.contains('oauth-note')) {
        note = document.createElement('p');
        note.className = 'field__msg oauth-note';
        note.setAttribute('role', 'status');
        row.after(note);
      }
      note.textContent = `${name} sign-in is coming soon. Please use your email for now.`;
    });
  }
}
```

CSS 한 줄 추가 (`auth.css`)

```
.oauth-note { margin-top: var(--sp-2); text-align: center; }
```

글자 크기 · 색은 기존 `.field__msg`(작은 회색 글)를 함께 써서 따로 정하지 않았다.

최신 문법으로 쓰면 (공부용 — 실제 코드는 위)

```
export function initOAuthPlaceholders(root = document) {
  root.querySelectorAll('.oauth[data-provider]:not([data-oauth-ready])').forEach((button) => {
    button.dataset.oauthReady = 'on';
    button.addEventListener('click', () => {
      const { provider } = button.dataset;
      const name = OAUTH_NAMES[provider] ?? provider;
      const row = button.closest('.oauth-row') ?? button.parentElement;
      let note = row.nextElementSibling;
      if (!note?.classList.contains('oauth-note')) {
        note = Object.assign(document.createElement('p'), { className: 'field__msg oauth-note', role: 's
tatus' });
        row.after(note);
      }
      note.textContent = `${name} sign-in is coming soon. Please use your email for now.`;
    });
  });
}
```

최신 문법	실제 코드	뜻
`const { provider } = button.dataset`	`const provider = button.dataset.provider`	구조 분해 — 칸을 같은 이름의 변수로 꺼냄
`OAUTH_NAMES[provider] ?? provider`	`OAUTH_NAMES[provider] || provider`	없으면 대신 쓸 값 (여기서는 둘의 결과가 같음 — 이름이 빈 글자일 일이 없음)
`!note?.classList.contains(…)`	`!note || !note.classList.contains(…)`	`note` 가 없거나, 안내 줄이 아니면
`Object.assign(요소, { … })`	`className = …` · `setAttribute('role', …)` 를 한 줄씩	여러 속성을 한 번에 넣기

문법 풀이

import 문법	뜻
`export function`	`join.js` · `home.js` 가 가져다 씀 (`OAUTH_NAMES` 는 `export` 가 없어 이 파일 안에서만 씀)

식	뜻
`'.oauth[data-provider]'`	`oauth` 클래스이면서 `data-provider` 속성이 있는 요소
`button.dataset.provider`	HTML 의 `data-provider="google"` → `'google'`
`OAUTH_NAMES[provider] || provider`	표에 있으면 보기 좋은 이름(`SoundCloud`), 없으면 받은 글자 그대로
`button.closest('.oauth-row')`	버튼에서 **위로** 올라가며 가장 가까운 버튼 줄
`row.nextElementSibling`	버튼 줄 **바로 다음** 요소 — 안내 줄이 이미 있으면 그것
`document.createElement('p')` · `row.after(note)`	새 `<p>` 를 만들어 버튼 줄 바로 뒤에 넣음
`` `${name} sign-in …` ``	템플릿 문자열 — `${ }` 안의 값을 글자에 끼움 (2015년 문법, 지금은 기본)

4. 흐름

1. 범위 안의 소셜 버튼을 모두 찾는다 (이미 연결한 버튼은 건너뜀).
2. 누르면 — 서비스 이름을 정하고, 버튼 줄 바로 아래에 안내 줄이 없으면 만든다.

3. 안내 줄의 글을 그 서비스 문구로 바꾼다.

5. 왜 이렇게

선택	이유
`alert` 대신 화면 안 안내 한 줄 (템플릿은 둘 다 가능)	`alert` 는 화면을 멈추고 확인 버튼을 눌러야 한다. 안내 한 줄은 하던 일을 방해하지 않는다
안내 줄을 하나만 두고 글만 바꿈	여러 번 눌러도 줄이 쌓이지 않는다
`role="status"`	화면 읽기 프로그램이 새 안내를 읽어 준다 (방해 없이 — `polite`)
"Please use your email for now."	막혔다는 말만 하지 않고 지금 할 수 있는 일을 알려 준다
같은 버튼에 두 번 연결하지 않음 (`data-oauth-ready`)	10 과 같은 이유
이름 표(`OAUTH_NAMES`)	`soundcloud` → `SoundCloud` 처럼 대소문자를 바르게. 표에 없는 서비스가 추가돼도 오류 없이 그 이름으로 보인다
CSS 는 기존 `.field__msg` 를 함께 씀	다른 칸 아래 문구와 같은 크기 · 색이 된다. 추가한 것은 여백 · 가운데 정렬 한 줄뿐

6. 시험 (실제 join.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 8개)

시험	확인한 것
연결	버튼 6개(가입 3 + 로그인 모달 3) 모두 연결, 한 번 더 불러도 중복 없음
누르기 전	안내 줄 없음
Google	줄 바로 아래 `<p class="field__msg oauth-note" role="status">` + Google 문구
Discord	안내 줄은 그대로 1개, 글만 Discord 로
SoundCloud	"SoundCloud" 표기
페이지 이동	없음 (`/join.html` 그대로)
모양	가운데 정렬, 위 여백 8px
표에 없는 서비스	`github` 그대로 표시, 오류 없음

8/8 통과. 위 캡처는 같은 방식으로 SoundCloud 를 누른 뒤 찍었다.

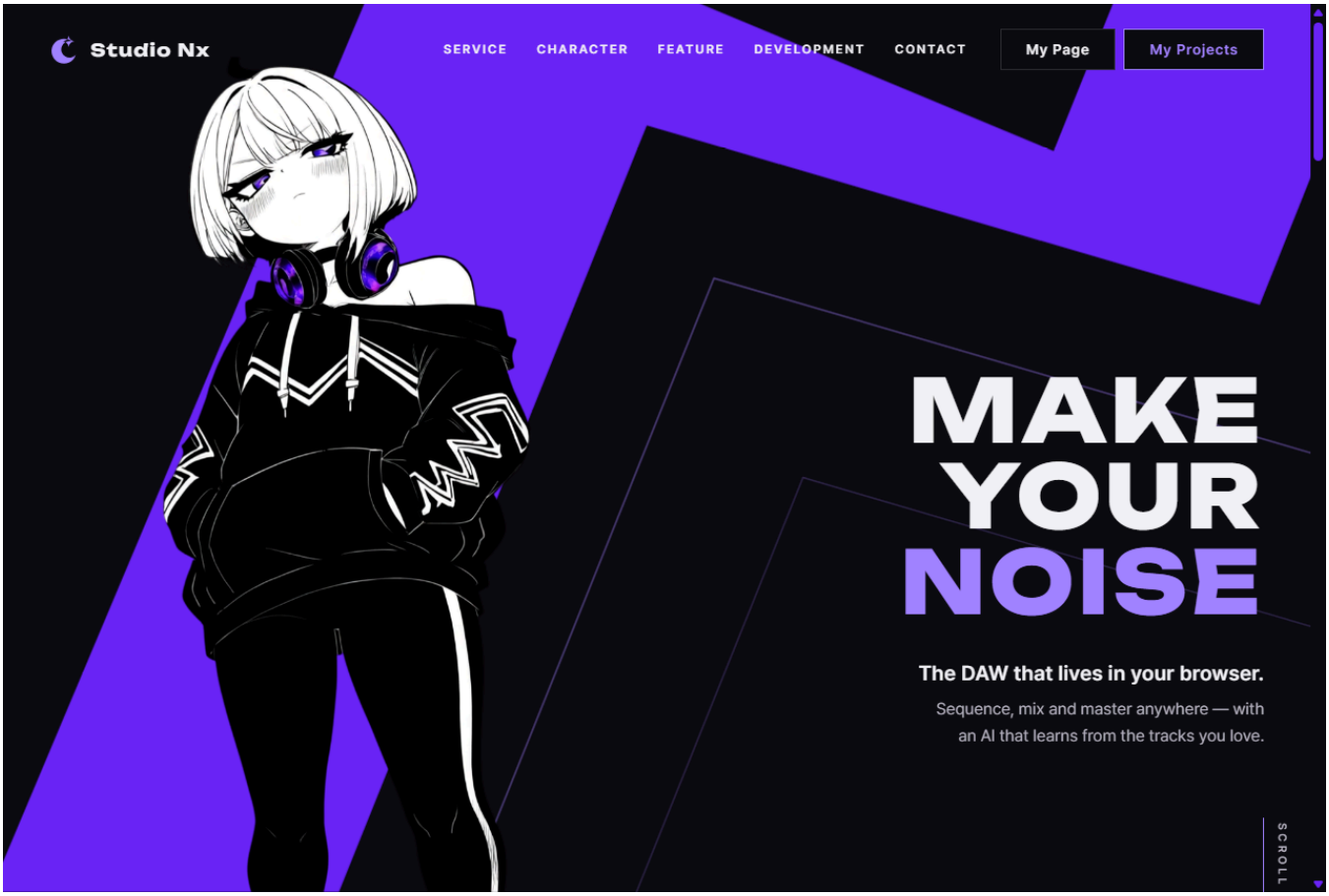
JS 12 — `initAuthState()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-105 · FR-109
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	11 `initOAuthPlaceholders()` · 13 `initLoginForm()`

1. 하는 일

페이지를 열 때 **로그인 상태를 확인**해서 `<html>` 에 표시한다. 그러면 `auth.css` 가 헤더 오른쪽 버튼을 바꾼다.

상태	`<html data-auth>`	헤더 오른쪽 버튼
확인 중 (서버 답을 기다림, 보통 0.1초 안)	`"pending"`	**비어 있음** (자리만 차지)
로그인 안 함	속성 없음	**Sign In · Join Us**
로그인함	`"in"`	**My Page · My Projects**



돌려주는 값: 로그인 상태면 `true`, 아니면 `false` (`Promise`).

2. 언제 쓰나

모든 페이지가 열릴 때 한 번 부른다 (이미 각 페이지 스크립트에 들어 있음).

페이지	부르는 곳
home.html	`home.js`
myPage.html · myProject.html · userInfo.html	`mypage.js` · `myproject.js` · `userinfo.js`
privacy.html	`privacy.js`

자동 로그인과의 관계 (FR-109): "Keep me signed in" 으로 로그인했으면 브라우저를 닫았다 열어도 리프레시 쿠키(3일)가 남아 있다 → 이 함수가 재발급에 성공 → My Page · My Projects.

페이지를 열 때 일어나는 일

순서	일어나는 일
①	`data-auth="pending"` → 헤더 버튼이 잠깐 안 보임
②	`isLoggedIn()` → `POST /api/v1/auth/refresh` (백엔드 33)
③-a	200 (쿠키 있음) → `data-auth="in"` → My Page · My Projects. 받은 토큰은 메모리에 남아 다음 API 요청에 씀
③-b	401 (쿠키 없음) → 속성 지움 → Sign In · Join Us

3. 코드

import (파일 맨 위)

```
import { isLoggedIn } from './auth.js';
```

`auth-ui.js` 의 첫 import 로 **이번에 추가했다**. 이 함수가 쓴다 — P1-02 의 06 `isLoggedIn()`.

함수

```
export async function initAuthState() {
  const root = document.documentElement;
  root.dataset.auth = 'pending';
  const loggedIn = await isLoggedIn();
  if (loggedIn) {
    root.dataset.auth = 'in';
  } else {
    root.removeAttribute('data-auth');
  }
  return loggedIn;
}
```

템플릿은 `export function` 이었고, 서버 답을 기다려야 해서 **`async` 로 바꿨다**. 부르는 쪽(`initAuthState();`)은 그대로 두어도 된다 — 기다리지 않고 다음 줄로 넘어가고, 답이 오면 헤더가 바뀐다.

CSS (이미 있음, `auth.css`)

```
:root:not([data-auth="in"]) [data-auth-show="in"],
:root[data-auth="in"] [data-auth-show="out"] { display: none; }
```

```
:root[data-auth="pending"] [data-auth-show] { visibility: hidden; }
```

```
<!-- header.html -->
<button … data-auth-show="out">Sign In</button>   <a … data-auth-show="out">Join Us</a>
<a href="myPage.html" data-auth-show="in">My Page</a>   <a href="myProject.html" data-auth-show="in">My Projects</a>
```

최신 문법으로 쓰면 (공부용 — 실제 코드는 위)

```
export async function initAuthState() {
  const root = document.documentElement;
  root.dataset.auth = 'pending';
  const loggedIn = await isLoggedIn();
  root.toggleAttribute('data-auth', loggedIn);
  if (loggedIn) root.dataset.auth = 'in';
  return loggedIn;
}
```

최신 문법	실제 코드	뜻
`root.toggleAttribute('data-auth', loggedIn)`	`if … else root.removeAttribute('data-auth')`	조건이 거짓이면 속성 지우기 (참이면 빈 속성을 붙이고, 다음 줄이 `'in'` 으로 덮음)

이 함수는 원래 짧아서 최신 문법과 차이가 작다. 실제 코드의 `if … else` 가 읽기 쉽다.

문법 풀이

import 문법	뜻
`import { isLoggedIn } from './auth.js'`	같은 폴더의 `auth.js` 가 `export` 한 함수를 가져옴

식	뜻
`document.documentElement`	페이지의 `<html>` 요소
`root.dataset.auth = 'pending'`	`<html data-auth="pending">` 로 만듦
`async function` · `await isLoggedIn()`	서버 답이 올 때까지 이 함수 안에서만 기다림 (페이지의 다른 동작은 멈추지 않음) — Promise 설명
`root.removeAttribute('data-auth')`	속성 자체를 지움 → CSS 의 "로그인 안 함" 규칙이 적용

4. 흐름

1. `<html data-auth="pending">` — 헤더 버튼을 잠깐 숨긴다.
2. 로그인 상태를 확인한다 (메모리 토큰이 없으면 재발급 시도).
3. 로그인이면 `"in"`, 아니면 속성을 지운다.
4. `true` / `false` 를 돌려준다.

5. 왜 이렇게

선택	이유
먼저 `pending` 으로 숨김	바로 "Sign In · Join Us" 를 보여 줬다가 0.1초 뒤 "My Page" 로 바뀌면 **깜빡인다**. 확인하는 동안은 버튼 자리만 비워 둔다 (`visibility: hidden` — 자리는 그대로라 헤더가 흔들리지 않음)
로그아웃이면 속성을 **지움** (`'out'` 이 아님)	CSS 는 "속성이 없으면 로그아웃" 으로 짜여 있다. JS 가 꺼져 있거나 오류가 나도 Sign In · Join Us 는 보인다
판단은 06 `isLoggedIn()` 에 맡김	토큰 · 쿠키 · 재발급 규칙은 P1-02 · P1-05 에 있다. 이 함수는 결과를 화면에 옮기기만 한다
`async` + 결과를 돌려줌 (템플릿은 일반 함수)	마이페이지처럼 "로그인돼 있어야 다음 일을 하는" 페이지는 `if (await initAuthState())` 로 기다릴 수 있다. 헤더만 바꾸는 페이지는 기다리지 않아도 된다

참고: 로그인하지 않은 방문자는 페이지를 열 때마다 `/auth/refresh` 가 401 을 받는다. 정상 흐름이지만, 개발자 도구의 네트워크 탭에는 빨간 401 이 한 줄씩 보인다 (백엔드 33 "쿠키 없음").

6. 시험 (실제 home.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 9개)

실제 서버(Sentry 를 끈 8001 번)에 **가입 · 로그인을 실제로 한 뒤** 확인했다. 시험 계정은 끝난 뒤 DB 에서 지웠다.

시험	확인한 것
페이지를 연 뒤 (로그인 안 함)	`data-auth` 없음, 헤더에 보이는 버튼 = Sign In · Join Us
부르자마자	`data-auth="pending"`, 헤더 버튼이 하나도 안 보임
답을 받은 뒤	`false`, 속성 없음
가입 → 로그인 → 메모리 토큰 지움 → 다시 부름	`true`, `data-auth="in"`, 헤더 = My Page · My Projects, 재발급으로 메모리 토큰도 생김
로그아웃 → 다시 부름	`false`, 속성 없음, 헤더 = Sign In · Join Us

"보이는 버튼" 은 실제 계산된 스타일(`display` · `visibility`)로 셌다 — `auth.css` 가 실제로 적용되는지까지 확인한 것이다. 9/9 통과. 위 캡처는 시험 끝에 로그인 상태로 찍었다.

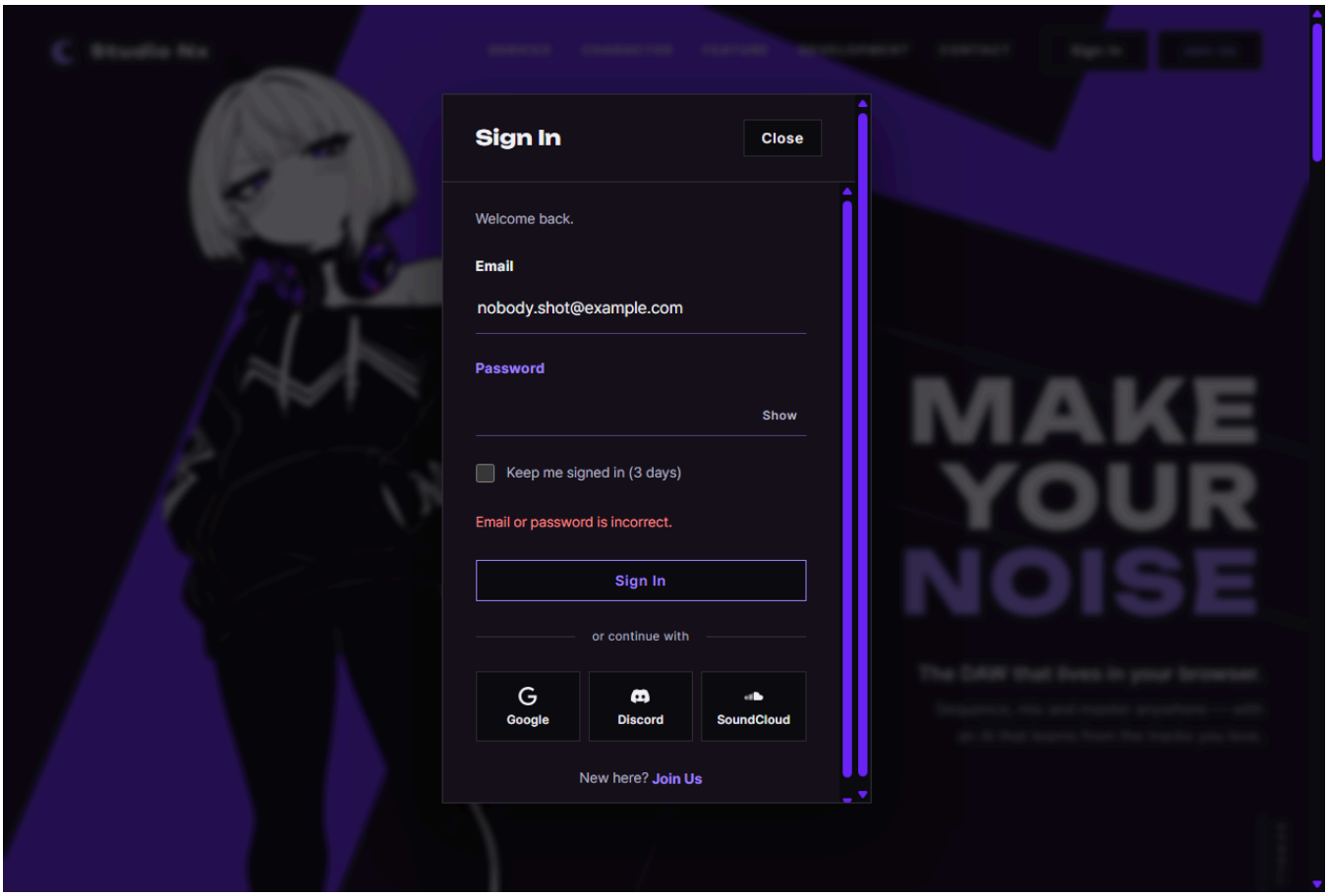
JS 13 — `initLoginForm()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-103 · FR-109
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	12 initAuthState() · 14 initLoginModalFromHash()

1. 하는 일

로그인 모달의 **Sign In** 버튼을 누르면 서버에 로그인하고, 결과에 따라 화면을 바꾼다. **이 함수로 사람이 화면에서 직접 로그인할 수 있게 됐다.**

경우	화면
맞음 · 주소에 `next` 없음 (home.html 에서 Sign In)	모달이 닫히고 헤더가 **My Page · My Projects** 로
맞음 · 주소에 `next` 있음 (`home.html#login?next=%2FmyPage.html`)	그 페이지(`/myPage.html`)로 이동
틀림 · 없는 이메일	모달 안 빨간 글 "Email or password is incorrect.", **비밀번호 칸만 비우고** 거기에 커서
정지 계정 (비밀번호 맞음)	빨간 글 "This account is disabled." (서버 문구)
빈 칸	브라우저가 "이 입력란을 작성하세요" — 서버로 보내지 않음



2. 언제 쓰나

로그인 모달이 있는 페이지가 열릴 때 한 번 부른다.

페이지	부르는 곳	성공하면
home.html	`home.js` — `initLoginForm({ onSuccess: null })`	기본 동작 (위 표)
join.html	`join.js` — 같음	기본 동작. 17 `initJoinForm` 때 `onSuccess` 로 "홈으로" 를 넣을 예정

누르면 일어나는 일

순서	일어나는 일	관련
①	빈 칸 · 이메일 모양을 **브라우저가** 먼저 검사 (HTML `required` · `type="email"`)	
②	Sign In 버튼 비활성 (두 번 누르기 방지), 오류 글 숨김	
③	`POST /api/v1/auth/login {email, password, remember}`	JS 02 · 백엔드 32
④-성공	`setAccessToken(access_token)` (메모리), 리프레시 쿠키는 서버가 심음	JS 01
⑤-성공	주소의 `next` 확인 → 이동 또는 모달 닫고 헤더 바꿈	JS 05 `parseLoginHash`
④-실패	오류 글 표시, 비밀번호 칸 비우고 커서	
⑥	버튼 다시 켬	

3. 코드

import (파일 맨 위)

```
import { api, setAccessToken } from './api.js';
import { isLoggedIn, parseLoginHash } from './auth.js';
```

`api` · `setAccessToken` · `parseLoginHash` 를 이번에 추가했다. 이 함수가 셋 다 쓴다 (`isLoggedIn` 은 12 가 씀).

함수

```
export function initLoginForm(options = {}) {
  const form = document.getElementById('login-form');
  if (!form || form.dataset.loginReady === 'on') return;
  form.dataset.loginReady = 'on';

  const email = document.getElementById('login-email');
  const password = document.getElementById('login-password');
  const remember = document.getElementById('login-remember');
  const error = document.getElementById('login-error');
  const submit = form.querySelector('button[type="submit"]');

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    error.hidden = true;
    submit.disabled = true;
    try {
      const data = await api.post(
```

```
        '/auth/login',
        { email: email.value, password: password.value, remember: remember.checked },
        { auth: false },
      );
      setAccessToken(data.access_token);
      const next = parseLoginHash().next;
      if (options.onSuccess) {
        options.onSuccess(next);
      } else if (next) {
        location.href = next;
      } else {
        const modal = document.getElementById('login-modal');
        if (modal && modal.open) modal.close();
        document.documentElement.dataset.auth = 'in';
        form.reset();
      }
    } catch (err) {
      if (err.status === 401 || err.status === 422) {
        error.textContent = 'Email or password is incorrect.';
      } else {
        error.textContent = err.message || 'Something went wrong. Please try again.';
      }
      error.hidden = false;
      password.value = '';
      password.focus();
    } finally {
      submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용 — 실제 코드는 위)

```
export function initLoginForm({ onSuccess } = {}) {
  const form = document.getElementById('login-form');
  if (!form || form.dataset.loginReady) return;
  form.dataset.loginReady = 'on';
  const [email, password, remember, error] =
    ['login-email', 'login-password', 'login-remember', 'login-error'].map((id) => document.getElementBy
Id(id));
  const submit = form.querySelector('[type="submit"]');

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    error.hidden = true;
    submit.disabled = true;
    try {
      const { access_token } = await api.post('/auth/login',
        { email: email.value, password: password.value, remember: remember.checked }, { auth: false });
      setAccessToken(access_token);
      const { next } = parseLoginHash();
      if (onSuccess) return onSuccess(next);
      if (next) return location.assign(next);
      document.getElementById('login-modal')?.close();
      document.documentElement.dataset.auth = 'in';
      form.reset();
```

```
    } catch (err) {
      error.textContent = [401, 422].includes(err.status)
        ? 'Email or password is incorrect.'
        : err.message ?? 'Something went wrong. Please try again.';
      error.hidden = false;
      password.value = '';
      password.focus();
    } finally {
      submit.disabled = false;
    }
  });
}
```

최신 문법	실제 코드	뜻
`function f({ onSuccess } = {})`	`function f(options = {}) + options.onSuccess`	인자 객체에서 바로 꺼냄 (구조 분해)
`const [a, b] = [...].map(…)`	한 줄에 하나씩 `const a = …`	여러 변수를 한 번에
`const { access_token } = await …`	`const data = await … + data.access_token`	응답에서 칸 꺼내기
`modal?.close()`	`if (modal && modal.open) modal.close()`	있을 때만
`[401, 422].includes(err.status)`	`err.status === 401 || err.status === 422`	둘 중 하나인지
`err.message ?? '…'`	`err.message || '…'`	없으면 대신 쓸 글 (빈 글자일 때는 결과가 다름 — 실제 코드는 빈 글자도 대신 씀)

문법 풀이

import 문법	뜻
`import { api, setAccessToken } from './api.js'`	같은 폴더 `api.js` 에서 두 이름을 가져옴

식	뜻
`form.addEventListener('submit', …)`	폼을 보낼 때(버튼 클릭 · Enter). 브라우저 검사(`required`)를 **통과해야만** 이 일이 일어남
`event.preventDefault()`	브라우저가 페이지를 새로 여는 원래 전송을 막음 — 대신 JS 가 보냄
`try { … } catch (err) { … } finally { … }`	실패하면 `catch`, 성공 · 실패와 상관없이 마지막에 `finally`
`err.status`	`ApiError` 의 HTTP 상태 번호
`{ auth: false }`	로그인 요청에는 토큰을 붙이지도, 재발급하지도 않음
`modal.open` · `modal.close()`	`<dialog>` 가 열려 있는지 · 닫기
`form.reset()`	폼의 입력을 처음 상태로
`location.href = next`	그 주소로 이동

4. 흐름

1. 폼이 없거나 이미 연결했으면 끝.
2. 보낼 때 — 오류 숨김, 버튼 끔.
3. 로그인 요청 → 성공하면 토큰 저장 → `onSuccess` / `next` 로 이동 / 모달 닫고 헤더 바꿈.
4. 실패하면 오류 글 · 비밀번호 비움 · 커서.
5. 버튼 다시 켬.

5. 왜 이렇게

선택	이유
401 · 422 는 같은 문구로	서버(29)와 같은 원칙 — 무엇이 틀렸는지 알려 주지 않는다. 422(형식 틀림)도 화면에서는 같은 문구
403 · 그 밖은 서버 문구 그대로	정지 계정("This account is disabled.")은 비밀번호가 맞은 본인에게만 오는 답이라 그대로 보여 준다. 네트워크 오류도 `api.js` 의 문구로
실패하면 **비밀번호만** 비움	대부분 비밀번호를 잘못 친 것이다. 이메일은 다시 치지 않게 남긴다
전송 중 버튼 끔	두 번 눌러 요청이 두 번 가는 것을 막는다
`onSuccess` 가 없으면 기본 동작 (템플릿은 페이지마다 넘기라고 함)	home.html 의 `onSuccess` 가 `null` 이어도 바로 쓸 수 있다. 다른 동작이 필요한 페이지(join.html)만 넘기면 된다
`next` 는 `parseLoginHash()` 로	JS 05 가 `safeNextPath` 로 검사한 값이라, 바깥 사이트로 보내지는 일이 없다
헤더는 재확인 없이 `data-auth="in"`	방금 로그인에 성공했으니 `/auth/refresh` 를 다시 부를 필요가 없다

6. 시험 (실제 home.html 에서 실제 서버로 · 12개)

시험 계정을 실제 가입 API 로 만들고, 끝난 뒤 DB 에서 지웠다.

시험	확인한 것
연결	`home.js` 가 페이지를 열 때 이미 연결함
틀린 비밀번호	전송 중 버튼 꺼짐 → 오류 "Email or password is incorrect.", 이메일 남음 · 비밀번호 빈 칸 · 커서가 비밀번호 칸 · 버튼 다시 켜짐, 토큰 없음 · 모달 열림
없는 이메일	같은 문구
빈 칸	`/auth/login` 요청이 하나도 나가지 않음 (브라우저 검사)
성공 (`next` 없음)	오류 숨김 · 모달 닫힘 · `data-auth="in"` · 폼 비움, 메모리 토큰 있음, 재발급 200 (쿠키도 생김)
성공 (`#login?next=%2FmyPage.html%3Ftab%3Dusage`)	`/myPage.html?tab=usage` 로 이동하려 함 (시험에서는 브라우저의 Navigation API 로 이동만 막고 목적지를 읽음)
`onSuccess` 를 준 경우	`onSuccess('/myPage.html?tab=usage')` 로 불림
두 번 부름	중복 연결 없음

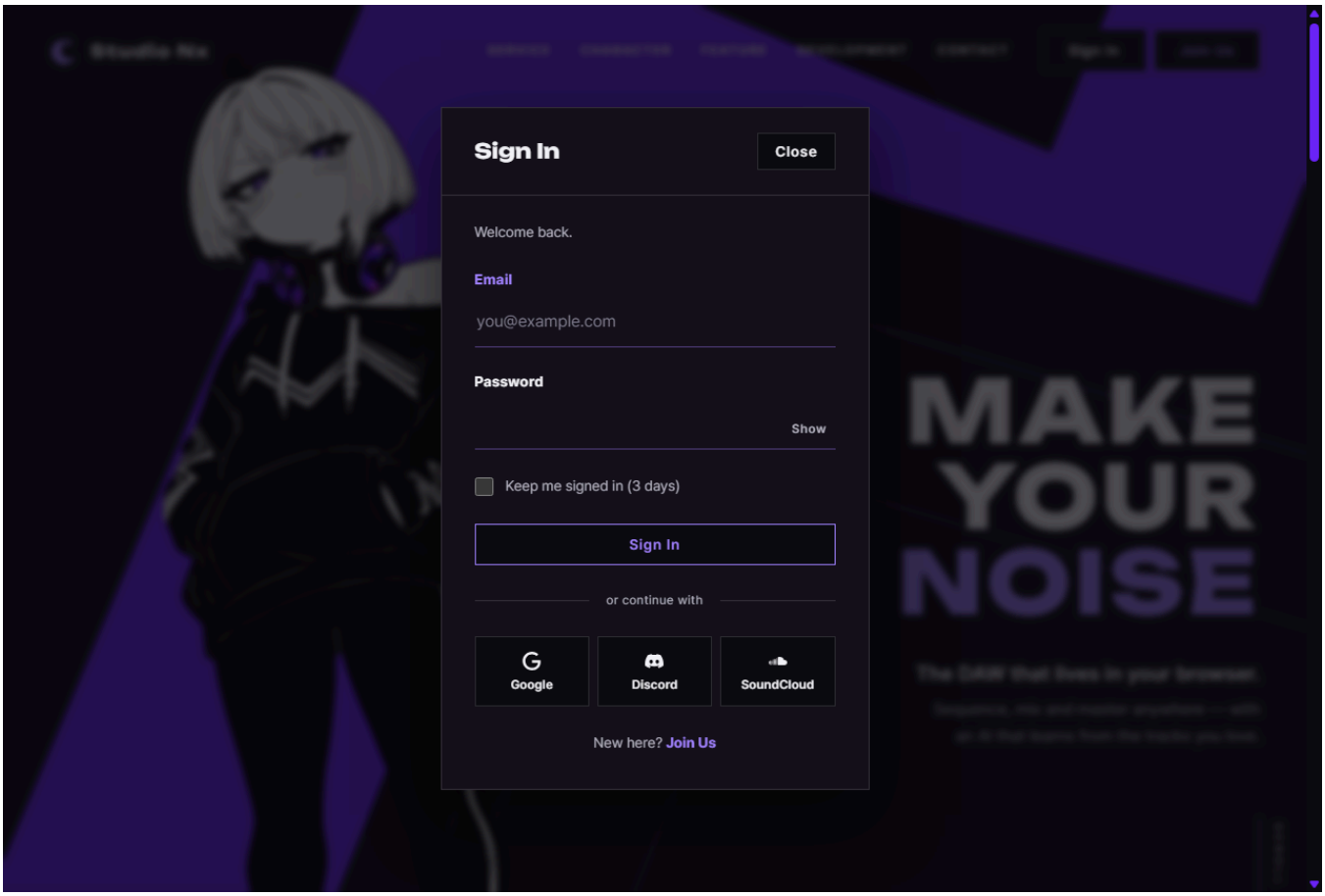
12/12 통과. 위 캡처는 없는 이메일 · 틀린 비밀번호로 보낸 뒤 찍었다.

JS 14 — `initLoginModalFromHash()`

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-103 (로그인 필요 페이지 → 로그인 → 원래 페이지)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	13 `initLoginForm()` · 15 `initDuplicateChecks()`

1. 하는 일

주소가 `home.html#login…` 이면 **로그인 모달을 자동으로 연다.** 모달을 닫으면 주소에서 `#login…` 을 지운다.



들어온 주소	결과
`home.html`	모달 닫힘 (평소대로)
`home.html#login`	모달 열림
`home.html#login?next=%2FmyProject.html%3Fsort%3Dtitle%23new`	모달 열림 — 로그인하면 `/myProject.html?sort=title#new` 로 (13)
`home.html#features`	모달 닫힘 (그 구역으로 스크롤만)

2. 언제 쓰나

로그인이 필요한 페이지가 로그인 안 된 사람을 홈으로 보낼 때 이 주소를 쓴다. **P1-02 · P1-05 · P1-07 이 이어지는 흐름의 가운데 고리**다.

순서	화면	일어나는 일	관련
①	myProject.html (로그인 안 함)	`requireLogin()` → 재발급 401 → `home.html#login?next=…` 로 이동	JS 07 · JS 04
②	home.html	**이 함수** — 모달 자동 열림	
③	로그인 모달	로그인 성공 → 주소의 `next` 로 이동	JS 13 · JS 05
④	myProject.html	`requireLogin()` → 재발급 200 → 그대로	

로그인하지 않고 **Close** 를 누르면 주소가 `home.html` 로 바뀐다. 새로 고침해도 모달이 다시 열리지 않는다.

3. 코드

import (`home.js` 맨 위)

```
import { initAuthState, initLoginForm, initPasswordToggles, initOAuthPlaceholders } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { parseLoginHash } from '../core/auth.js';
```

`parseLoginHash` 줄을 이번에 추가했다. 이 함수는 그것만 쓴다.

함수

```
export function initLoginModalFromHash() {
  const modal = document.getElementById('login-modal');
  if (!modal) return;

  const openIfLoginHash = () => {
    if (parseLoginHash().open && !modal.open) modal.showModal();
  };

  modal.addEventListener('close', () => {
    if (location.hash.startsWith('#login')) {
      history.replaceState(null, '', location.pathname + location.search);
    }
  });
  window.addEventListener('hashchange', openIfLoginHash);
  openIfLoginHash();
}
```

`home.js` 아래쪽에서 이미 `includePartials()`(모달 HTML 끼우기) 다음에 부르고 있다 — 모달이 있어야 찾을 수 있으므로 순서가 중요하다.

최신 문법으로 쓰면 (공부용 — 실제 코드는 위)

```
export function initLoginModalFromHash() {
  const modal = document.getElementById('login-modal');
```

```
  if (!modal) return;
  const openIfLoginHash = () => parseLoginHash().open && !modal.open && modal.showModal();
  modal.addEventListener('close', () => {
    if (location.hash.startsWith('#login')) history.replaceState(null, '', location.pathname + location.
search);
  });
  addEventListener('hashchange', openIfLoginHash);
  openIfLoginHash();
}
```

최신 문법	실제 코드	뜻
`조건 && 조건 && 실행()`	`if (조건 && 조건) 실행()`	`&&` 를 "참이면 다음을 실행" 으로 쓰는 짧은 습관 — 읽기 어려워 실제 코드는 `if`
`addEventListener(…)` (앞에 `window.` 없음)	`window.addEventListener(…)`	전역 함수는 `window.` 를 생략할 수 있음

이 함수는 최신 전용 문법이 거의 없다 — 차이는 "짧게 쓰는 습관" 정도다.

문법 풀이

import 문법	뜻
`import { parseLoginHash } from '../core/auth.js'`	`../` = 한 폴더 위(`static/js/`) → `core/auth.js`

식	뜻
`const openIfLoginHash = () => { … }`	이름 붙인 화살표 함수 — 아래에서 두 번(처음 · `hashchange`) 씀
`modal.showModal()` · `modal.open`	`<dialog>` 를 모달로 열기 · 열려 있는지
`modal.addEventListener('close', …)`	모달이 **어떤 이유로든** 닫힐 때 (Close 버튼 · Esc · 로그인 성공 후 `close()`)
`location.hash.startsWith('#login')`	주소의 `#` 부분이 `#login` 으로 시작하는지
`history.replaceState(null, '', 새 주소)`	페이지를 다시 열지 않고 **주소 글자만** 바꿈. 방문 기록도 새로 쌓지 않음
`window.addEventListener('hashchange', …)`	페이지 안에서 `#…` 부분만 바뀔 때

4. 흐름

1. 모달이 없으면 끝.
2. 모달이 닫힐 때 → 주소에서 `#login…` 을 지우도록 연결.
3. 주소의 `#` 가 바뀔 때 → 다시 확인하도록 연결.
4. 지금 주소를 확인 → `#login` 이면 연다.

5. 왜 이렇게

선택	이유
`next` 를 따로 저장하지 않음 (템플릿은 "저장해 두었다가")	모달이 열려 있는 동안 주소에 `#login?next=…` 가 그대로 있다. 13 이 로그인 성공 순간에 `parseLoginHash()` 로 직접 읽으므로, 저장 장소를 하나 더 둘 필요가 없다
닫으면 `#login…` 제거	그대로 두면 새로 고침할 때마다 모달이 다시 뜬다
`replaceState` (`location.hash = ''` 아님)	`hash` 를 바꾸면 방문 기록이 한 칸 쌓이고, 주소 끝에 `#` 가 남는다. `replaceState` 는 둘 다 없다

선택	이유
`hashchange` 도 받음 (템플릿에 없음)	이미 home.html 을 보고 있을 때 `#login` 링크를 누르거나 주소창에 쳐도 모달이 열린다
`!modal.open` 확인	이미 열린 모달에 `showModal()` 을 또 부르면 브라우저 오류가 난다
바깥 주소 `next` 는 걸러짐	판단은 JS 05 → JS 03 `safeNextPath` 가 한다. 모달은 열리지만 `next` 는 `null` 이라, 로그인하면 홈에 머문다

6. 시험 (실제 home.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 8개)

시험	확인한 것
`home.html#login?next=%2FmyPage.html` 로 열기	모달 열림, `parseLoginHash()` = `{open: true, next: "/myPage.html"}`
닫기	주소가 `/home.html` (해시 없음)
페이지 안에서 `#login` 으로	모달 열림 (`hashchange`), 닫으면 다시 제거
`#features`	모달 안 열림
`#login?next=https%3A%2F%2Fevil.com`	모달 열림, `next` 는 `null`
`home.html` (해시 없이) 로 열기	모달 닫혀 있음

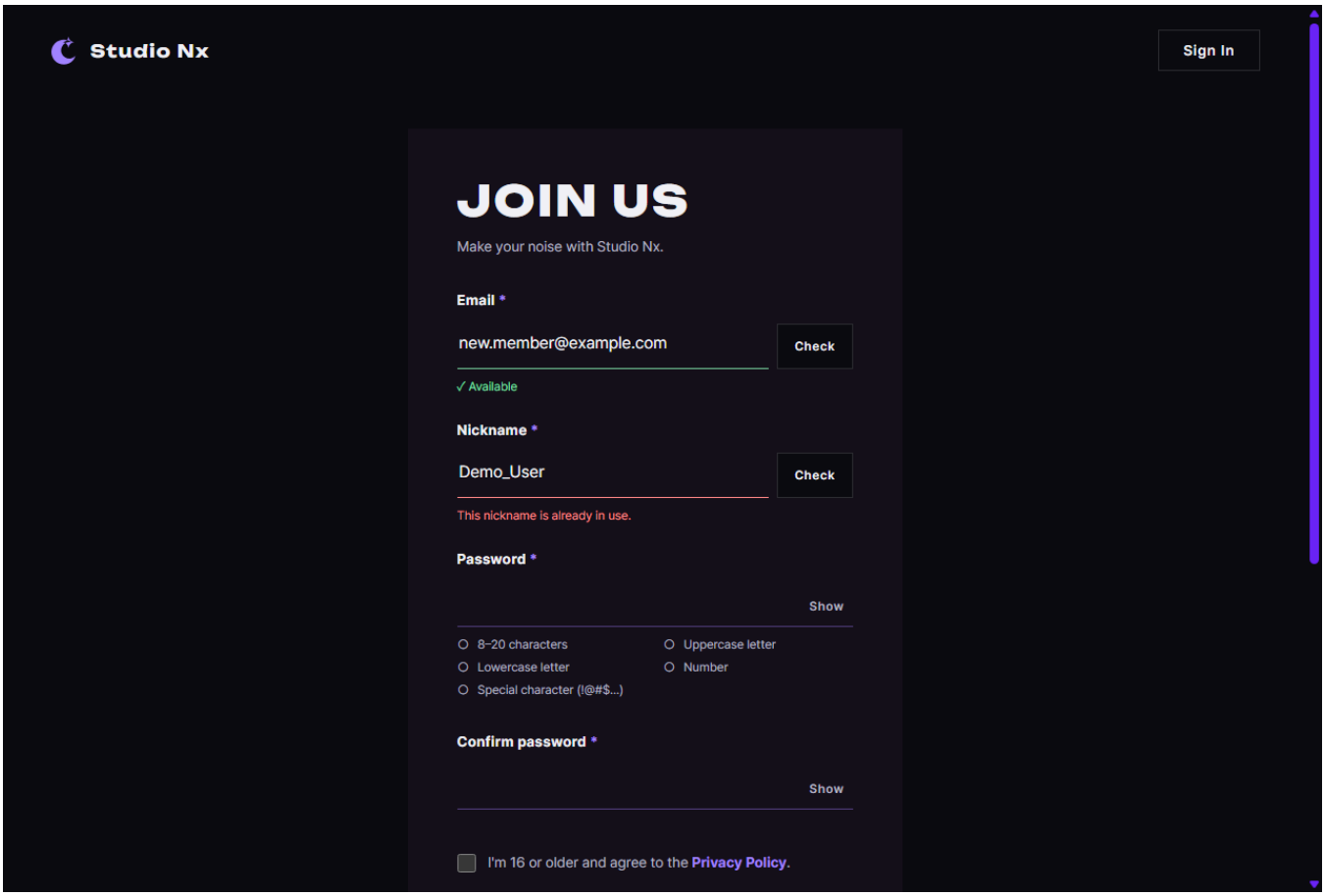
8/8 통과. 위 캡처는 `home.html#login?next=%2FmyProject.html%3Fsort%3Dtitle%23new` 로 열고, 첫 접속 로딩 화면이 끝난 뒤 찍었다.

JS 15 — `initDuplicateChecks()`

항목	내용
파일	`studionx/frontend/static/js/pages/join.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-101 (실시간 중복 확인) · FR-108 (탈퇴 후 3일)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	14 `initLoginModalFromHash()` · 16 `initPasswordRules()`

1. 하는 일

join.html 의 이메일 · 닉네임 칸이 **이미 쓰이고 있는지** 서버에 물어 칸 아래에 보여 준다. **Check** 버튼을 누르거나, 입력을 멈추고 0.5초가 지나면 묻는다.



입력	칸 아래
(빈 칸) · `abc` · `kim@naver`	빨강 "Enter a valid email address." — **서버에 묻지 않음**
없는 이메일	잠깐 "Checking…" → 초록 "✓ Available"
가입된 이메일 (`DEMO@Example.com` 처럼 대소문자가 달라도)	빨강 "This email is already in use."
탈퇴 후 3일 안인 이메일	빨강 "You can sign up again after Oct 9." (날짜는 **사용자 시간대**)
닉네임 1자 · 21자	빨강 "Use 2–20 characters." — 서버에 묻지 않음
가입된 닉네임 (`Demo_User` ↔ `demo_user`)	빨강 "This nickname is already in use."

입력을 고치면 결과가 바로 지워진다 — 다시 확인해야 가입할 수 있다.

2. 언제 쓰나

join.html 이 열릴 때 `join.js` 가 한 번 부른다. 묻는 곳은 P1-04 에서 만든 API 다.

칸	API	서버
이메일	`GET /api/v1/auth/check-email?email=`	백엔드 23
닉네임	`GET /api/v1/auth/check-nickname?nickname=`	백엔드 24

확인 결과는 가입 전송(17 `initJoinForm`)이 쓴다. "✓ Available" 이 나온 값은 칸에 `data-checked="그 값"` 으로 남는다. 가입 버튼을 누를 때 지금 값과 같은지 보고, 다르면(확인 안 했거나 고친 뒤) 가입을 막는다.

3. 코드

import (`join.js` 맨 위)

```
import { initLoginForm, initPasswordToggles, initOAuthPlaceholders, setFieldState } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { api } from '../core/api.js';
```

`api` 줄을 이번에 추가했다. 이 함수는 `api`(서버에 묻기)와 09 `setFieldState`(칸 상태)를 쓴다.

함수

```
/** 중복 확인 칸마다 다른 것 — 주소 · 문구 · 서버에 묻기 전 형식 검사 */
const CHECKS = {
  email: {
    path: '/auth/check-email?email=',
    taken: 'This email is already in use.',
    invalid: 'Enter a valid email address.',
    isValid: (input) => input.value.trim() !== '' && input.checkValidity(),
  },
  nickname: {
    path: '/auth/check-nickname?nickname=',
    taken: 'This nickname is already in use.',
    invalid: 'Use 2–20 characters.',
    isValid: (input) => input.value.trim().length >= 2 && input.value.trim().length <= 20,
  },
};

export function initDuplicateChecks() {
  for (const kind of Object.keys(CHECKS)) {
    const rule = CHECKS[kind];
    const field = document.getElementById(`join-${kind}-field`);
    const input = document.getElementById(`join-${kind}`);
    const button = document.querySelector(`[data-check="${kind}"]`);
    if (!field || !input) continue;
    let timer = null;
```

```
    const check = async () => {
      clearTimeout(timer);
      delete field.dataset.checked; // 이번 확인을 통과해야 다시 남김
      const value = input.value.trim();
      if (!rule.isValid(input)) {
        setFieldState(field, 'error', rule.invalid);
        return;
      }
      setFieldState(field, null, 'Checking…');
      let result;
      try {
        result = await api.get(rule.path + encodeURIComponent(value), { auth: false });
      } catch (err) {
        if (input.value.trim() !== value) return; // 그사이 입력이 바뀜 — 옛 응답은 버림
        setFieldState(field, 'error', err.status === 422 ? rule.invalid : err.message);
        return;
      }
      if (input.value.trim() !== value) return;
      if (result.available) {
        field.dataset.checked = value;
        setFieldState(field, 'ok', '✓ Available');
      } else if (result.reason === 'REJOIN_TOO_SOON') {
        const day = new Date(result.available_at).toLocaleDateString('en-US', { month: 'short', day: 'nu
meric' });
        setFieldState(field, 'error', `You can sign up again after ${day}.`);
      } else {
        setFieldState(field, 'error', rule.taken);
      }
    };

    input.addEventListener('input', () => {
      delete field.dataset.checked; // 값이 바뀌면 다시 확인해야 함
      setFieldState(field, null);
      clearTimeout(timer);
      if (rule.isValid(input)) timer = setTimeout(check, 500);
    });
    if (button) button.addEventListener('click', check);
  }
}
```

최신 문법으로 쓰면 (공부용 — 실제 코드는 위, 바뀌는 부분만)

```
for (const [kind, rule] of Object.entries(CHECKS)) { … }
…
const { available, reason, available_at } = await api.get(…);
…
button?.addEventListener('click', check);
```

최신 문법	실제 코드	뜻
`for (const [kind, rule] of Object.entries(CHECKS))`	`for (const kind of Object.keys(CHECKS))` + `const rule = CHECKS[kind]`	열쇠와 값을 한 번에 꺼냄
`const { available, reason, available_at } = await …`	`result = await …` + `result.available`	응답에서 칸 꺼내기 (구조 분해)
`button?.addEventListener(…)`	`if (button) button.addEventListener(…)`	버튼이 있을 때만

문법 풀이

import 문법	뜻
`import { api } from '../core/api.js'`	`pages/` 에서 한 폴더 위 → `core/api.js`

식	뜻
`` `join-${kind}-field` ``	`kind` 가 `'email'` 이면 `'join-email-field'`
`input.checkValidity()`	브라우저의 기본 검사(`type="email"` · `required` · `maxlength`)를 통과하면 `true`
`setTimeout(check, 500)` · `clearTimeout(timer)`	0.5초 뒤에 실행하도록 예약 · 예약 취소. 입력할 때마다 취소하고 다시 예약 → **입력을 멈춘 뒤** 0.5초
`encodeURIComponent(value)`	주소에 넣을 수 있게 바꿈 (`+` → `%2B`, `@` → `%40`) — `a+b@x.com` 이 공백으로 바뀌는 것을 막음
`delete field.dataset.checked`	`data-checked` 속성을 지움
`new Date(…).toLocaleDateString('en-US', { month: 'short', day: 'numeric' })`	UTC 시각을 **이 컴퓨터의 시간대**로 바꿔 `Oct 9` 모양으로

4. 흐름

Check 를 누를 때 (또는 자동 확인)

1. 예약된 자동 확인 취소, 지난 통과 표시 지움.
2. 형식이 틀리면 빨간 문구로 끝 (서버에 묻지 않음).
3. "Checking…" → 서버에 묻는다.
4. 답이 오기 전에 입력이 바뀌었으면 **버린다**.
5. 사용 가능이면 초록 + `data-checked`, 탈퇴 3일 안이면 날짜 문구, 사용 중이면 빨간 문구.

입력할 때

1. 지난 결과를 지운다.
2. 형식이 맞으면 0.5초 뒤 자동 확인을 예약한다.

5. 왜 이렇게

선택	이유
서버에 묻기 전에 형식 검사	`abc` 처럼 틀린 값은 서버까지 갈 필요가 없다. 서버도 422 로 막지만(23 · 24), 요청을 아낀다
자동 확인은 형식이 맞을 때만	`kim@na` 까지 쳤을 때 빨간 "Enter a valid…" 가 뜨면 입력 중인 사람을 재촉하는 셈이다. 다 쳤을 때만 묻는다. Check 버튼을 누르면 형식 문구도 보여 준다
0.5초 기다림 (템플릿 권장)	글자마다 묻지 않는다. `kim@naver.com` 13자를 칠 때 요청 13번 → 1번
늦게 온 답은 버림	`demo@…` 을 확인하는 중에 다른 값으로 고치면, 늦게 온 "사용 중" 이 새 값에 붙을 수 있다. 답이 왔을 때 칸의 값이 물어본 값과 같은지 본다
확인할 때마다 `data-checked` 를 먼저 지움	시험에서 찾은 빈틈: 통과 → (입력 이벤트 없이 값이 바뀜) → 사용 중이 나와도 옛 통과 표시가 남았다. 이제 이번 확인을 통과해야만 남는다
날짜는 사용자 시간대	서버는 UTC(`…Z`)로 준다. 한국 사용자에게는 한국 날짜로 보여야 한다
칸마다 다른 것은 `CHECKS` 표로	이메일 · 닉네임의 동작은 같고 주소 · 문구 · 형식 규칙만 다르다. 같은 코드를 두 번 쓰지 않는다

6. 시험 (실제 join.html 에서 실제 서버로 · 15개)

데모 계정(`demo@example.com` · `demo_user`)과, 시험용으로 **탈퇴 1일 된 계정**을 DB 에 넣고 확인했다 (탈퇴 계정은 끝난 뒤 지움).

시험	확인한 것
빈 칸 · `abc` + Check	"Enter a valid email address.", 서버 요청 0번
없는 이메일	누르자마자 "Checking..." → "✓ Available", `data-checked` 남음
`DEMO@Example.com`	"This email is already in use.", `data-checked` 없음
탈퇴 1일 이메일	"You can sign up again after Oct 9."
입력	바로 결과 지움, 0.3초에는 아직 안 물음, 0.5초 지나면 자동 확인 → ✓
`auto.ch` 처럼 덜 친 입력	자동 확인 안 함 (빨간 글 없음, 요청 0번)
확인 중에 값을 바꿈	옛 값의 "사용 중" 이 새 값에 붙지 않음
닉네임 `a` · `DEMO_USER` · `" fresh_nick "` · 탈퇴 1일 닉네임	2–20자 문구 · 사용 중 · ✓(`data-checked="fresh_nick"`) · 재가입 날짜 문구

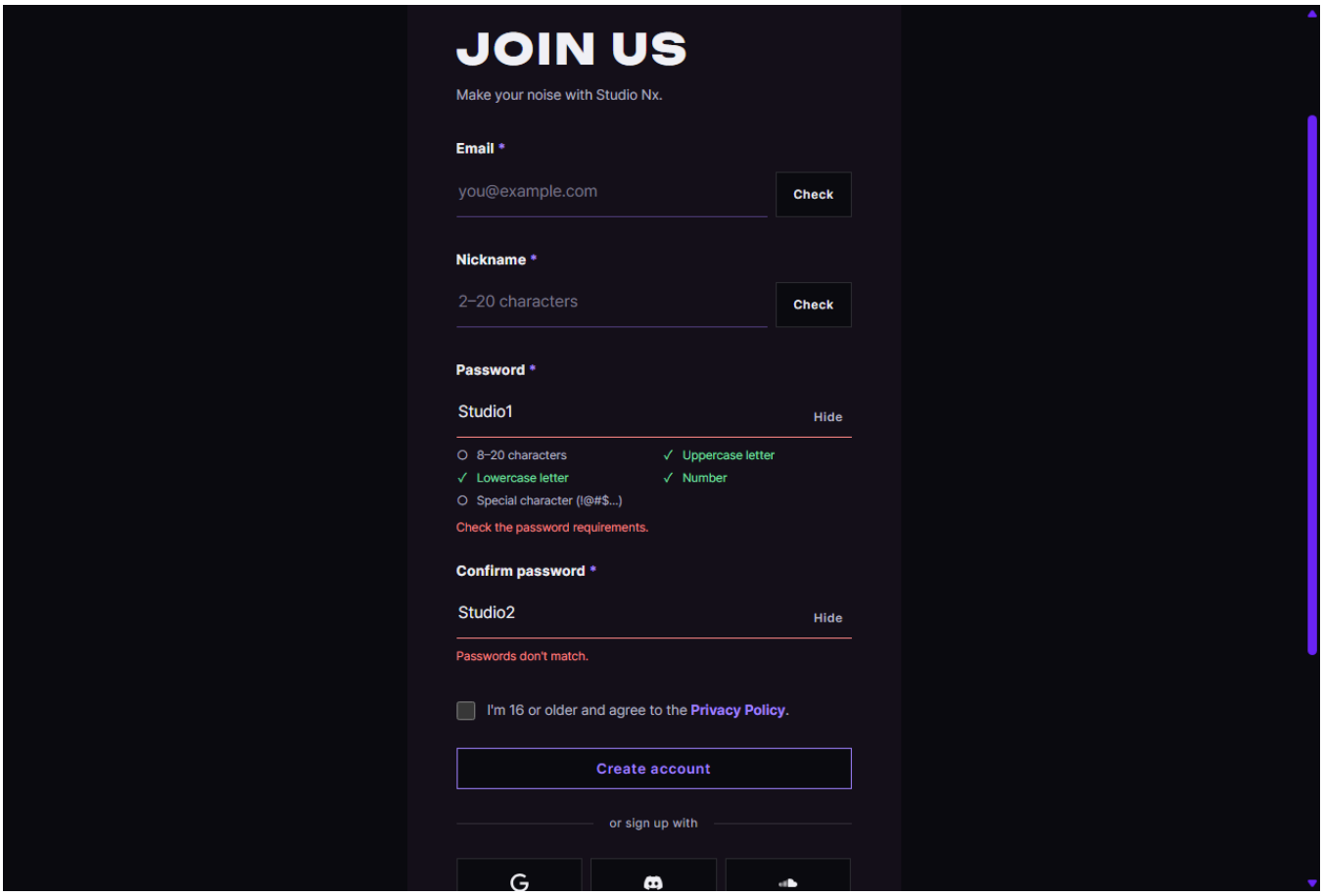
처음 돌렸을 때 14/15 — 위 §5 의 빈틈(`data-checked` 가 남음)을 고친 뒤 15/15 통과.

JS 16 — `initPasswordRules()`

항목	내용
파일	`studionx/frontend/static/js/pages/join.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-102 (비밀번호 조건을 입력할 때마다 ✓)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	15 `initDuplicateChecks()` · 17 `initJoinForm()`

1. 하는 일

join.html 의 비밀번호를 칠 때마다 아래 **조건 목록의 ○ 를 ✓(초록)** 로 바꾸고, 틀린 점을 칸 아래에 알려 준다. 비밀번호 확인 칸이 다르면 알려 준다.



입력	조건 목록	칸 아래
`a`	소문자만 ✓	(없음)
`Studio1` (입력 중)	대문자 · 소문자 · 숫자 ✓, 8–20자 · 기호 ○	(없음 — 입력 중에는 재촉하지 않음)
`Studio1` → 칸을 벗어남	같음	빨강 "Check the password requirements."
`Studio1!`	모두 ✓	(빨간 글 사라짐)
`Studio1!한` · `Studio 1!`		**바로** 빨강 "Use only English letters, numbers and symbols."

입력	조건 목록	칸 아래
확인 칸 `Studio2` (비밀번호만큼 침)		빨강 "Passwords don't match."
확인 칸을 맞게 고침		**바로** 사라짐

조건 목록은 화면 읽기 프로그램용 글자도 함께 바뀐다 — "Uppercase letter, met" / "..., not met".

2. 언제 쓰나

join.html 이 열릴 때 `join.js` 가 한 번 부른다.

규칙	어디에	같은 규칙인 곳
`PASSWORD_RULES` (5개) · `PASSWORD_ALLOWED`	`join.js` 위쪽 (템플릿에 이미 있음)	서버 백엔드 14 `check_password_rules` — 시험으로 같은 기호 모음임을 확인함

화면의 ✓ 는 **안내**다. 가입 버튼을 누르면 17 `initJoinForm` 이 같은 규칙으로 한 번 더 막고, 서버도 다시 검사한다.

3. 코드

import (`join.js` 맨 위 — 15 와 같음, 이번에 추가한 것 없음)

```
import { initLoginForm, initPasswordToggles, initOAuthPlaceholders, setFieldState } from '../core/auth-ui.js';
…
```

이 함수는 **`setFieldState`**(09)를 쓰고, 규칙은 같은 파일의 `PASSWORD_RULES` · `PASSWORD_ALLOWED` 를 쓴다.

규칙 (템플릿에 있던 것)

```
export const PASSWORD_RULES = {
  length: (pw) => pw.length >= 8 && pw.length <= 20,
  upper: (pw) => /[A-Z]/.test(pw),
  lower: (pw) => /[a-z]/.test(pw),
  number: (pw) => /[0-9]/.test(pw),
  special: (pw) => /[!-/:-@[-`{-~]/.test(pw),
};
export const PASSWORD_ALLOWED = /^[!-~]*$/;
```

함수

```
export function initPasswordRules() {
  const password = document.getElementById('join-password');
  const confirm = document.getElementById('join-password2');
  const passwordField = document.getElementById('join-password-field');
  const confirmField = document.getElementById('join-password2-field');
  const items = document.querySelectorAll('#join-password-rules li[data-rule]');
  if (!password || !confirm) return;

  const allMet = () => {
```

```
    for (const name of Object.keys(PASSWORD_RULES)) {
      if (!PASSWORD_RULES[name](password.value)) return false;
    }
    return true;
  };

  const showRules = () => {
    for (const item of items) {
      const rule = PASSWORD_RULES[item.dataset.rule];
      const met = rule ? rule(password.value) : false;
      item.classList.toggle('is-met', met);
      const state = item.querySelector('[data-rule-state]');
      if (state) state.textContent = met ? ', met' : ', not met';
    }
  };

  const checkConfirm = (force) => {
    if (confirm.value === '') {
      setFieldState(confirmField, null);
    } else if (confirm.value === password.value) {
      setFieldState(confirmField, null);
    } else if (force || confirm.value.length >= password.value.length) {
      setFieldState(confirmField, 'error', "Passwords don't match.");
    }
  };

  password.addEventListener('input', () => {
    showRules();
    if (!PASSWORD_ALLOWED.test(password.value)) {
      setFieldState(passwordField, 'error', 'Use only English letters, numbers and symbols.');
    } else {
      setFieldState(passwordField, null);
    }
    if (confirm.value !== '') checkConfirm(true); // 확인 칸을 이미 쳤으면 바로 다시 비교
  });
  password.addEventListener('blur', () => {
    if (password.value !== '' && PASSWORD_ALLOWED.test(password.value) && !allMet()) {
      setFieldState(passwordField, 'error', 'Check the password requirements.');
    }
  });
  confirm.addEventListener('input', () => checkConfirm(false));
  confirm.addEventListener('blur', () => checkConfirm(true));
  showRules();
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const allMet = () => Object.values(PASSWORD_RULES).every((rule) => rule(password.value));
…
item.querySelector('[data-rule-state]')?.replaceChildren(met ? ', met' : ', not met');
…
const rule = PASSWORD_RULES[item.dataset.rule];
const met = rule?.(password.value) ?? false;
```

최신 문법	실제 코드	뜻
`Object.values(…).every(fn)`	`for … of Object.keys(…)` 로 돌다가 하나라도 틀리면 `return false`	모두 참인지
`rule?.(password.value)`	`rule ? rule(password.value) : …`	함수가 있을 때만 부르기
`… ?? false`	`: false` (삼항 연산자의 뒷부분)	없으면 `false`
`el?.replaceChildren(글)`	`if (state) state.textContent = 글`	있을 때만 글자 바꾸기

문법 풀이

import 문법	뜻
`export const PASSWORD_RULES`	17 `initJoinForm` 도 같은 규칙을 씀 (같은 파일)

식	뜻
`/[A-Z]/.test(pw)`	정규식 — `pw` 안에 대문자가 하나라도 있으면 `true`
`/^[!-~]*$/`	처음(`^`)부터 끝(`$`)까지 모든 글자가 `!`(33) ~ `~`(126) 범위 — 영문 · 숫자 · 키보드 기호. 한글 · 공백은 범위 밖
`item.classList.toggle('is-met', met)`	`met` 이 참이면 클래스 붙이고, 거짓이면 뗌 (2015년 무렵부터 있던 기본 기능)
`PASSWORD_RULES[item.dataset.rule]`	`<li data-rule="upper">` → `PASSWORD_RULES.upper` 함수
`'input'` · `'blur'`	글자가 바뀔 때마다 · 칸에서 커서가 나갈 때
`checkConfirm(force)`	`force` 가 참이면 길이와 상관없이 "다름" 을 보여 줌

4. 흐름

비밀번호를 칠 때: 조건 목록 ✓/○ → 허용 안 되는 글자면 바로 빨강, 아니면 빨간 글 지움 → 확인 칸이 이미 있으면 다시 비교. **비밀번호 칸을 벗어날 때:** 조건이 하나라도 안 맞으면 "Check the password requirements." **확인 칸을 칠 때:** 같으면 지움, 비밀번호 길이만큼 쳤는데 다르면 "다름". **확인 칸을 벗어날 때:** 다르면 길이와 상관없이 "다름".

5. 왜 이렇게

선택	이유
조건 미달은 칸을 **벗어날 때** 알림 (템플릿 그대로)	치는 중에 "조건이 안 맞음" 을 계속 보여 주면 재촉하는 것 같다. 조건 목록 ✓ 가 이미 진행 상황을 보여 준다
허용 안 되는 글자는 **바로** 알림	한글 입력 상태로 치는 실수를 빨리 알아챈다. 계속 치면 바로잡기가 더 어렵다
확인 칸은 비밀번호만큼 쳤을 때부터 "다름"	한 글자 칠 때마다 "다름" 이 뜨면 안 된다. 다 친 뒤에야 비교가 의미 있다
맞으면 바로 지움	고친 순간 확인받는 느낌을 준다
비밀번호를 고치면 확인 칸도 다시 비교	시험에서 찾은 경우: 확인까지 맞게 친 뒤 비밀번호 끝에 한 글자를 더하면, 확인 칸이 "맞음" 인 채로 남았다
✓ 는 CSS 가 그림 (`li.is-met::before`)	JS 는 클래스만 바꾼다 (09 와 같은 원칙)

선택	이유
화면 읽기 글자(`, met`)도 바꿈	색과 기호(✓)만으로는 보지 못하는 사람이 알 수 없다
규칙은 서버와 같게	화면은 통과인데 서버가 막는 일이 없게. 기호 모음이 같은지는 백엔드 14 시험으로 확인했다

6. 시험 (실제 join.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 17개)

시험	확인한 것
처음	모두 ○
`a` · `Abcdef1`	소문자만 ✓ / 대문자 · 소문자 · 숫자 ✓, 화면 읽기 글자 `, met` · `, not met`
입력 중 / 벗어남	입력 중 빨간 글 없음 / 벗어나면 "Check the password requirements."
`Abcdef1!`	모두 ✓, 빨간 글 사라짐, 벗어나도 오류 없음
한글 · 공백	바로 "Use only English letters, numbers and symbols."
24자	8–20자 ○
확인 칸	짧을 때 말 안 함 · 비밀번호만큼 쳤는데 다르면 "다름" · 같아지면 지움 · 짧은 채로 벗어나면 "다름" · 비밀번호를 고치면 다시 "다름" · 비우면 지움

처음 돌렸을 때 두 가지가 실패했다.

- **시험 쪽 문제:** 페이지 스크립트가 끝나기 전에 시험이 시작됐다. 시험 도구가 2.5초 기다리게 고쳤다.
- **실제 빈틈:** 비밀번호를 고쳐도 확인 칸을 다시 비교하지 않았다. §5 대로 고쳤다.

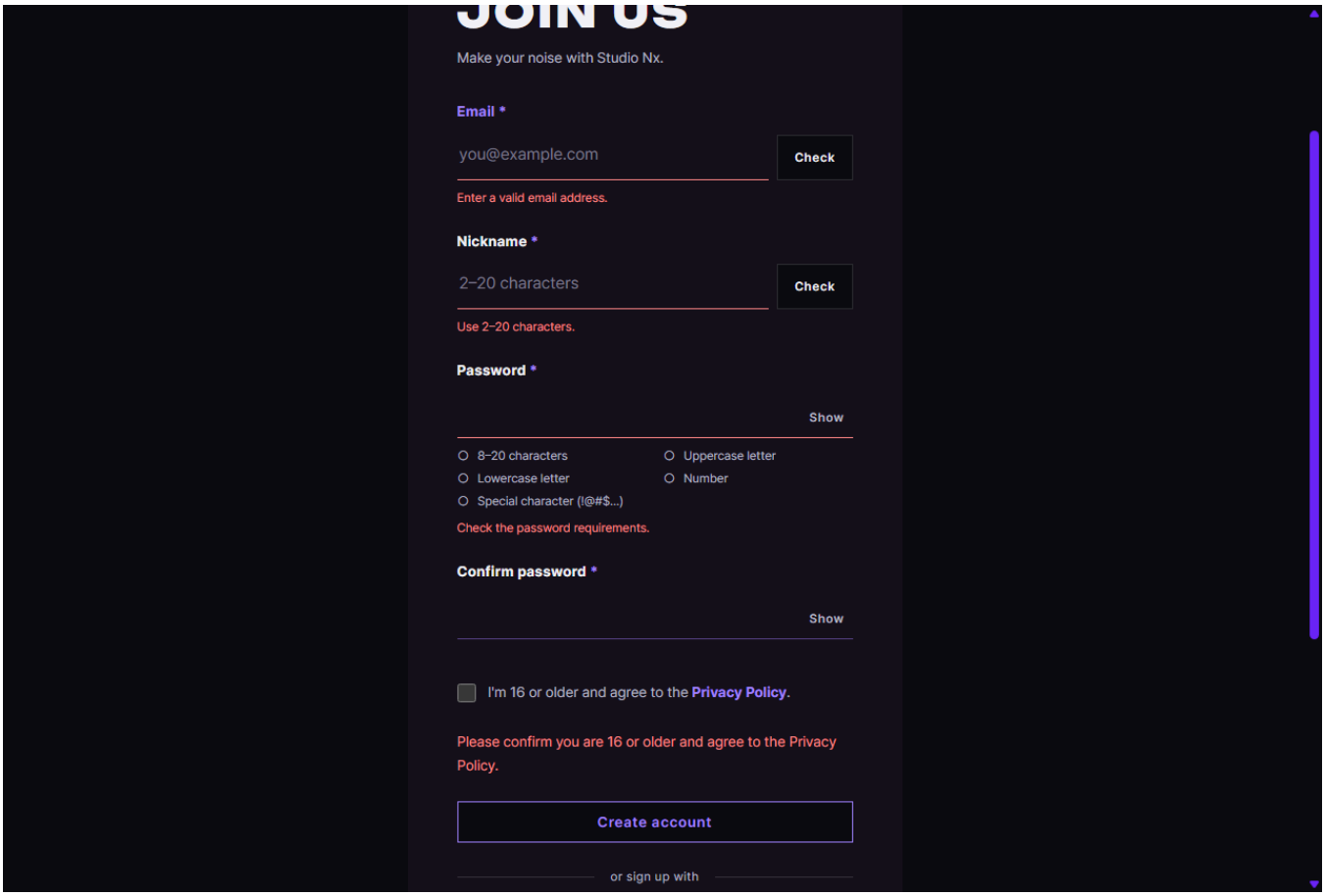
고친 뒤 17/17 통과. 앞의 09 · 10 · 11 · 15 시험도 다시 돌려 모두 통과했다.

JS 17 — `initJoinForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/join.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-101 · FR-102 · 처리방침 동의 (P1-07 마지막)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	16 `initPasswordRules()` · (P1-07 끝)

1. 하는 일

join.html 의 **Create account** 를 누르면 모든 칸을 검사하고, 맞으면 서버에 가입한 뒤 **로그인 모달이 열린 홈**으로 보낸다. **이 함수로 사람이 화면에서 직접 가입할 수 있게 됐다.**



경우	화면
빈 칸 · 형식 틀림	그 칸 아래 빨간 문구, 커서는 **첫 번째로 틀린 칸**. 서버에 보내지 않음
중복 확인 전에 누름 (입력 후 0.5초 안)	"Click Check to confirm this email is available."
비밀번호 조건 · 확인 칸	"Check the password requirements." · "Passwords don't match."
동의 체크 안 함	버튼 위 빨간 글 "Please confirm you are 16 or older and agree to the Privacy Policy."
모두 맞음	버튼 잠깐 꺼짐 → `home.html#login` 으로 이동 → 로그인 모달 자동 열림 (14)

경우	화면
확인 뒤 다른 사람이 먼저 가입 (서버 409)	그 칸에 "This email is already in use.", 커서 그 칸, 버튼 다시 켜짐

함께 바꾼 한 줄 — 가입 화면의 로그인 모달(이미 가입한 사람용)이 성공하면 홈으로 가게 했다.

```
initLoginForm({ onSuccess: (next) => { location.href = next || 'home.html'; } });
```

2. 언제 쓰나

join.html 이 열릴 때 `join.js` 가 한 번 부른다. 앞 함수들의 결과를 모아 최종 판단한다.

검사	근거	만든 곳
이메일 · 닉네임 형식	`CHECKS[kind].isValid`	15
이메일 · 닉네임 중복 확인 통과	칸의 `data-checked` 가 지금 값과 같은지	15
비밀번호 조건 · 허용 문자	`PASSWORD_RULES` · `PASSWORD_ALLOWED`	16
확인 칸	비밀번호와 같은지	16
동의	`#join-agree` 체크	join.html
서버	`POST /api/v1/auth/signup`	백엔드 21 · 22

가입 한 번의 전체 흐름 (P1-04 · 05 · 07)

순서	화면	일어나는 일
①	join.html	이메일 · 닉네임 → 0.5초 뒤 ✓ (15), 비밀번호 ✓ (16), 동의 체크
②	Create account	**이 함수** → `POST /auth/signup` → 201
③	home.html#login	로그인 모달 자동 열림 (14)
④	로그인	헤더 My Page · My Projects (13)

3. 코드

import (`join.js` 맨 위 — 15 와 같음, 이번에 추가한 것 없음)

```
import { initLoginForm, initPasswordToggles, initOAuthPlaceholders, setFieldState } from '../core/auth-ui.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { api } from '../core/api.js';
```

이 함수는 **`api`** · **`setFieldState`** 를 쓰고, 같은 파일의 `CHECKS`(15) · `PASSWORD_RULES` · `PASSWORD_ALLOWED` · `PRIVACY_VERSION` 을 쓴다.

함수 (길어서 세 부분으로 나눠 설명)

```
export function initJoinForm() {
  const form = document.getElementById('join-form');
  if (!form) return;
```

```
const fields = { email: …, nickname: …, password: …, password2: … };   // 칸(.field) 4개
const email = …, nickname = …, password = …, password2 = …, agree = …, error = …, submit = …;

const showFormError = (message) => {
  error.textContent = message;
  error.hidden = false;
};
const AGREE_MESSAGE = 'Please confirm you are 16 or older and agree to the Privacy Policy.';

// 동의를 체크하면 동의 안내를 바로 지움 (다른 안내는 그대로) — 사이트 시험(ai-note 39)에서 찾아 추가
agree.addEventListener('change', () => {
  if (agree.checked && error.textContent === AGREE_MESSAGE) error.hidden = true;
});
```

① 화면에서 먼저 검사 — 틀린 칸의 입력 요소 목록을 돌려줌

```
const findProblems = () => {
  const problems = [];
  for (const kind of ['email', 'nickname']) {
    const input = kind === 'email' ? email : nickname;
    const value = input.value.trim();
    if (!CHECKS[kind].isValid(input)) {
      setFieldState(fields[kind], 'error', CHECKS[kind].invalid);
      problems.push(input);
    } else if (fields[kind].dataset.checked !== value) {
      if (!fields[kind].classList.contains('is-error')) {
        setFieldState(fields[kind], 'error', `Click Check to confirm this ${kind} is available.`);
      }
      problems.push(input);
    }
  }
  let rulesMet = PASSWORD_ALLOWED.test(password.value);
  for (const name of Object.keys(PASSWORD_RULES)) {
    if (!PASSWORD_RULES[name](password.value)) rulesMet = false;
  }
  if (!rulesMet) {
    setFieldState(fields.password, 'error', 'Check the password requirements.');
    problems.push(password);
  }
  if (password2.value !== password.value) {
    setFieldState(fields.password2, 'error', "Passwords don't match.");
    problems.push(password2);
  }
  if (!agree.checked) {
    showFormError(AGREE_MESSAGE);
    problems.push(agree);
  }
  return problems;
};
```

② 서버가 막은 이유를 그 칸 아래에 (409 · 422)

```
const showServerError = (err) => {
  const details = err.details || {};
  if (err.code === 'EMAIL_TAKEN' || details.email) {
    delete fields.email.dataset.checked;
```

```
      setFieldState(fields.email, 'error', err.code === 'VALIDATION_ERROR' ? CHECKS.email.invalid : err.
message);
      email.focus();
    } else if (err.code === 'NICKNAME_TAKEN' || details.nickname) {
      delete fields.nickname.dataset.checked;
      setFieldState(fields.nickname, 'error', err.code === 'VALIDATION_ERROR' ? CHECKS.nickname.invalid
: err.message);
      nickname.focus();
    } else if (details.password) {
      setFieldState(fields.password, 'error', 'Check the password requirements.');
      password.focus();
    } else {
      showFormError(err.message || 'Something went wrong. Please try again.');
    }
  };
```

③ 보내기

```
  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    error.hidden = true;
    const problems = findProblems();
    if (problems.length > 0) {
      problems[0].focus();
      return;
    }
    submit.disabled = true;
    try {
      await api.post('/auth/signup', {
        email: email.value.trim(), nickname: nickname.value.trim(), password: password.value,
        privacy_agreed: true, privacy_version: PRIVACY_VERSION,
      }, { auth: false });
      location.href = 'home.html#login';
    } catch (err) {
      showServerError(err);
      submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const rulesMet = PASSWORD_ALLOWED.test(password.value)
  && Object.values(PASSWORD_RULES).every((rule) => rule(password.value));
…
const { code, message, details = {} } = err;
…
problems.at(0)?.focus();
```

최신 문법	실제 코드	뜻
`Object.values(…).every(fn)`	`for … of` 로 돌며 하나라도 틀리면 `false`	모두 참인지
`const { code, details = {} } = err`	`err.code` · `const details = err.details || {}`	꺼내면서 없으면 기본값
`problems.at(0)?.focus()`	`if (problems.length > 0) problems[0].focus()`	첫 번째가 있으면

문법 풀이

import 문법	뜻
(추가 없음)	15 에서 넣은 `api` 와, 처음부터 있던 `setFieldState` 를 씀

식	뜻
`<form … novalidate>` (join.html)	브라우저의 기본 검사 말풍선을 끔 — 이 함수가 칸 아래 문구로 직접 보여 줌 (13 의 로그인 폼은 브라우저 검사를 씀)
`const findProblems = () => { … return problems; }`	안쪽 함수 — 틀린 칸들을 모아 돌려줌
`problems.push(input)`	목록 끝에 넣기
`problems[0].focus()`	목록의 첫 번째 = 화면 위쪽부터 검사했으므로 **가장 위의 틀린 칸**
`` `Click Check to confirm this ${kind} is available.` ``	`kind` 가 `'email'` 이면 "… this email is available."
`err.code` · `err.details`	`ApiError` 가 서버 오류 JSON 에서 꺼내 둔 값
`location.href = 'home.html#login'`	홈으로 이동 + 14 가 로그인 모달을 엶

4. 흐름

1. 보낼 때 — 폼 전체 오류 글 숨김.
2. 화면에서 모든 칸 검사 → 틀린 칸이 있으면 문구를 붙이고 첫 칸으로 커서, 끝.
3. 버튼 끄고 서버에 가입 요청.
4. 성공 → `home.html#login`. 실패 → 그 칸(또는 폼 아래)에 문구, 버튼 다시 켬.

5. 왜 이렇게

선택	이유
모든 칸을 검사한 뒤 첫 칸으로 커서	하나씩 고치고 다시 누르게 하지 않는다. 틀린 곳을 한 번에 다 보여 준다
중복 확인을 통과한 값만 보냄 (`data-checked`)	템플릿 요구. 보내기 전에 "이미 사용 중" 을 알면 서버까지 갔다 돌아올 필요가 없다. 보통은 입력 후 0.5초 자동 확인(15)이 끝나 있어 따로 누를 일이 없다
이미 빨간 문구(예: 사용 중)가 있으면 그대로 둠	"Click Check…" 로 덮으면 왜 안 되는지가 사라진다
서버 409 · 422 를 그 칸에	확인과 가입 사이에 다른 사람이 먼저 가입하는 경우(백엔드 21 의 동시 가입)도 사용자는 어느 칸을 고칠지 안다
가입 후 자동 로그인 안 함 → 로그인 모달 (템플릿 선택지 중 하나)	가입과 로그인을 나눈 서버 설계(백엔드 22)와 맞춘다. "Keep me signed in" 을 직접 고를 수 있다
성공하면 버튼을 다시 켜지 않음	곧 다른 페이지로 간다. 그사이 또 누르는 것을 막는다
동의를 체크하면 동의 안내를 바로 지움	사이트 시험(ai-note 39)에서 찾은 빈틈: 체크했는데도 빨간 "Please confirm…" 이 다음 전송까지 남아 있었다. 서버 오류 같은 다른 안내는 지우지 않는다
가입 화면의 로그인 모달 → 홈으로	가입 화면에서 로그인한 사람이 가입 화면에 남을 이유가 없다. `next` 가 있으면 그쪽으로

6. 시험 (실제 join.html 에서 실제 서버로 · 10개)

시험	확인한 것
빈 채로 보냄	이메일 · 닉네임 · 비밀번호 문구, 동의 문구, 가입 요청 0번, 커서 = 이메일 칸
다 맞게 치고 0.5초 안에 보냄	"Click Check to confirm this email/nickname is available.", 요청 0번
자동 확인 뒤 보냄	둘 다 ✓ → 보내는 동안 버튼 꺼짐 → 요청 1번 → `/home.html#login` 으로 이동하려 함 (시험에서는 이동만 막음) → 서버에서 그 이메일이 이제 `TAKEN`
확인 통과로 꾸민 `demo@example.com` 으로 보냄	서버 409 → "This email is already in use.", 통과 표시 지움, 버튼 다시 켜짐, 커서 이메일 칸
가입 화면에서 로그인 모달로 로그인	`/home.html` 로 이동하려 함
동의 없이 보낸 뒤 체크 (사이트 시험 뒤 추가)	동의 안내가 바로 사라짐

처음 9/9 통과. 사이트 시험에서 동의 안내가 남는 빈틈을 찾아 고친 뒤 시험 1개를 더해 10/10 통과. 앞 함수들(09 ~ 16)의 시험도 다시 돌려 모두 통과했다. 시험에서 만든 계정은 끝난 뒤 지웠다 (데모 계정만 남김).